

Projeto Final em Go: Construindo uma Blockchain com Proof of Work e Ledger Persistente

Por Antônio Marcelo

1. Objetivo do projeto

Neste projeto final vamos construir uma **blockchain simples em Go**, utilizando o mecanismo de consenso **Proof of Work**, ou Prova de Trabalho.

Nossa blockchain será capaz de:

- Criar um bloco inicial, chamado **Genesis Block**
- Adicionar novos blocos
- Armazenar informações dentro dos blocos
- Calcular hashes utilizando SHA-256
- Ligar cada bloco ao bloco anterior
- Executar um pequeno processo de mineração usando Proof of Work
- Verificar se a blockchain continua válida
- Salvar a blockchain em um arquivo
- Recuperar automaticamente a blockchain quando o programa for iniciado novamente
- Continuar adicionando blocos a partir do último bloco salvo

O ponto mais importante deste projeto é a **persistência**.

Se nossa blockchain possuir:

```
Bloco 0  
Bloco 1  
Bloco 2  
Bloco 3
```

e desligarmos o programa, esses blocos não serão perdidos.

Quando executarmos novamente:

```
go run .
```

o programa abrirá o arquivo da blockchain, carregará os quatro blocos e continuará a partir do:

```
Bloco 4
```

Isso será feito através de um arquivo chamado:

```
ledger.json
```

2. O que é uma blockchain?

Uma blockchain pode ser imaginada como um **livro de registros digital**.

Esse livro é dividido em páginas.

Cada página representa um **bloco**.

Por exemplo:

BLOCO 1

Compra realizada por João

BLOCO 2

Venda realizada para Maria

BLOCO 3

Pagamento realizado por Carlos

A grande diferença é que cada página possui uma espécie de "impressão digital matemática" da página anterior.

Essa impressão digital é chamada de:

Hash

Temos então algo parecido com:

Bloco 1

Hash: ABC123

↓

Bloco 2

PreviousHash: ABC123

Hash: DEF456

↓

Bloco 3

PreviousHash: DEF456

Hash: XYZ789

Dessa maneira, os blocos ficam conectados.

3. O que é um Hash?

Um hash é uma espécie de impressão digital de uma informação.

Vamos utilizar o algoritmo:

SHA-256

Por exemplo, o texto:

Curso de Go

pode gerar um hash semelhante a:

15434af8237d76...

Uma pequena mudança:

produziria outro hash completamente diferente.

Por isso hashes são extremamente importantes para blockchains.

Em Go utilizaremos:

```
import "crypto/sha256"
```

4. Estrutura do projeto

Crie uma pasta chamada:

```
blockchain-go
```

Entre nela:

```
cd blockchain-go
```

Inicialize o módulo Go:

```
go mod init blockchain-go
```

Vamos utilizar inicialmente apenas um arquivo:

```
blockchain-go/
├── go.mod
└── main.go
```

Depois da primeira execução aparecerá:

```
blockchain-go/
├── go.mod
├── main.go
└── ledger.json
```

O arquivo `ledger.json` será criado automaticamente pelo nosso programa.

5. Estrutura de um bloco

Cada bloco terá os seguintes campos:

```
type Block struct {
    Index      int    `json:"index"`
    Timestamp  string `json:"timestamp"`
    Data       string `json:"data"`
    PreviousHash string `json:"previous_hash"`
    Hash       string `json:"hash"`
    Nonce      int    `json:"nonce"`
}
```

Vamos entender cada campo.

Index

Número do bloco.

0
1
2
3
4

O bloco zero será o Genesis Block.

Timestamp

Momento em que o bloco foi criado.

Exemplo:

2026-08-17T13:30:00-03:00

Data

Informação que queremos registrar.

Exemplo:

Produto vendido para cliente João

PreviousHash

Hash do bloco anterior.

É isso que cria a corrente entre os blocos.

Hash

Hash do próprio bloco.

Nonce

Número encontrado durante a mineração.

Ele será utilizado no Proof of Work.

6. Nossa blockchain

Nossa blockchain será simplesmente um slice de blocos:

```
var blockchain []Block
```

Podemos imaginá-lo como:

```
blockchain
├── Block 0
├── Block 1
├── Block 2
└── Block 3
```

7. Calculando o Hash

Criaremos a função:

```
func calculateHash(block Block) string {
    record := strconv.Itoa(block.Index) +
        block.Timestamp +
        block.Data +
        block.PreviousHash +
        strconv.Itoa(block.Nonce)

    hash := sha256.Sum256([]byte(record))

    return hex.EncodeToString(hash[:])
}
```

Observe que pegamos várias informações do bloco:

Index
Timestamp
Data
PreviousHash
Nonce

e juntamos tudo.

Depois usamos:

```
sha256.Sum256()
```

para calcular o hash.

8. O Proof of Work

Agora entra uma das partes mais interessantes do projeto.

Vamos determinar que um bloco somente será considerado minerado se seu hash começar com:

0000

Exemplo de hash inválido:

93a12f735818...

Outro inválido:

71bc92943232...

Até encontrarmos:

0000a73128c93...

Esse hash será aceito.

9. Para que serve o Nonce?

O computador precisa modificar alguma coisa para gerar hashes diferentes.

É aí que entra o:

Nonce

Começamos:

Nonce = 0

Calculamos:

Hash = 93abc...

Não começou com 0000.

Tentamos:

Nonce = 1

Resultado:

Hash = 8af31...

Também não.

Continuamos:

Nonce = 2

Nonce = 3

Nonce = 4

Nonce = 5

...

Até aparecer algo semelhante a:

Nonce = 42581

Hash = 0000a6319fc...

Encontramos!

O bloco foi minerado.

10. Criando a função de mineração

Podemos implementar dessa maneira:

```
func mineBlock(block *Block) {  
  
    target := strings.Repeat("0", difficulty)  
  
    fmt.Println("Minerando bloco...")  
  
    for {  
  
        block.Hash = calculateHash(*block)  
  
        if strings.HasPrefix(block.Hash, target) {  
            break  
        }  
  
        block.Nonce++  
    }  
  
    fmt.Println("Bloco minerado!")  
    fmt.Println("Nonce encontrado:", block.Nonce)  
    fmt.Println("Hash:", block.Hash)
```

```
}
```

Observe este loop:

```
for {
```

Ele continuará executando até encontrarmos o hash desejado.

Quando isso acontece:

```
break
```

encerra o loop.

11. Definindo a dificuldade

No início do programa colocaremos:

```
const difficulty = 4
```

Isso significa:

Hash precisa começar com 4 zeros.

Portanto:

```
0000xxxxxxxxxxxxxxxxxxxx
```

Para computadores utilizados em sala de aula, dificuldade 4 é suficiente para demonstrar o conceito.

Para deixar mais rápido podemos usar:

```
const difficulty = 3
```

O alvo seria:

```
000xxxxxxxxxxxxxxxxxxxx
```

12. Criando o Genesis Block

Toda blockchain precisa começar em algum lugar.

Esse primeiro bloco é chamado:

```
Genesis Block
```

Como não existe bloco anterior, usaremos:

```
PreviousHash = "0"
```

Nossa função será:

```
func createGenesisBlock() Block {  
    block := Block{  
        Index:      0,  
        Timestamp:  time.Now().Format(time.RFC3339),  
        Data:        "Genesis Block",  
        PreviousHash: "0",  
        Nonce:       0,  
    }  
}
```

```
        mineBlock(&block)

        return block
}
```

13. Criando novos blocos

Depois do Genesis Block podemos criar novos blocos.

A função recebe:

```
func createBlock(data string, previousBlock Block) Block
```

Precisamos do bloco anterior porque queremos acessar seu hash.

```
func createBlock(data string, previousBlock Block) Block {

    block := Block{
        Index:      previousBlock.Index + 1,
        Timestamp:   time.Now().Format(time.RFC3339),
        Data:        data,
        PreviousHash: previousBlock.Hash,
        Nonce:       0,
    }

    mineBlock(&block)

    return block
}
```

Observe:

```
PreviousHash: previousBlock.Hash
```

Aqui acontece a ligação da blockchain.

14. Visualmente

Imagine que temos:

```
BLOCO 0
Hash:
0000AAA
```

Criamos o bloco 1:

```
BLOCO 1

PreviousHash:
0000AAA

Hash:
0000BBB
```

Depois:

```
BLOCO 2

PreviousHash:
```


0000BBB

Hash:
0000CCC

Nossa corrente fica:

```
[ BLOCO 0 ]  
Hash 0000AAA  
    |  
    ▼  
[ BLOCO 1 ]  
PreviousHash 0000AAA  
Hash 0000BBB  
    |  
    ▼  
[ BLOCO 2 ]  
PreviousHash 0000BBB  
Hash 0000CCC
```

É daí que vem o nome:

Block + Chain

ou:

Cadeia de Blocos

15. O problema da memória

Se fizermos apenas:

```
var blockchain []Block
```

nossa blockchain ficará armazenada na memória RAM.

Quando encerramos o programa:

CTRL + C

ou fechamos o terminal, perdemos tudo.

Precisamos portanto salvar nossa blockchain.

16. Criando nosso Ledger

Vamos utilizar:

```
ledger.json
```

Ledger significa algo semelhante a:

Livro de registros
Livro razão
Registro de transações

Nossa blockchain inteira será armazenada nele.

Exemplo:

```
[
```

```

{
    "index": 0,
    "timestamp": "2026-08-17T13:30:00-03:00",
    "data": "Genesis Block",
    "previous_hash": "0",
    "hash": "0000822ab...",
    "nonce": 53822
},
{
    "index": 1,
    "timestamp": "2026-08-17T13:32:00-03:00",
    "data": "Venda realizada",
    "previous_hash": "0000822ab...",
    "hash": "000091ac...",
    "nonce": 17283
}
]

```

17. Salvando o ledger

Criaremos:

```

func saveBlockchain(blockchain []Block) error {
    data, err := json.MarshalIndent(blockchain, "", "  ")
    if err != nil {
        return err
    }
    err = os.WriteFile("ledger.json", data, 0644)
    if err != nil {
        return err
    }
    return nil
}

```

A função:

`json.MarshalIndent()`

transforma nossas structs Go em JSON.

Depois:

`os.WriteFile()`

grava os dados no arquivo.

18. Carregando a blockchain

Quando iniciarmos o programa precisamos verificar:

ledger.json existe?

Se existir:

Carrega blockchain

Se não existir:

Cria Genesis Block

Nossa função será:

```
func loadBlockchain() ([]Block, error) {  
  
    data, err := os.ReadFile("ledger.json")  
  
    if err != nil {  
        return nil, err  
    }  
  
    var blockchain []Block  
  
    err = json.Unmarshal(data, &blockchain)  
  
    if err != nil {  
        return nil, err  
    }  
  
    return blockchain, nil  
}
```

Aqui fazemos o contrário.

Marshal:

Go → JSON

Unmarshal:

JSON → Go

19. Detectando a primeira execução

Podemos verificar se o arquivo existe usando:

```
_, err := os.Stat("ledger.json")
```

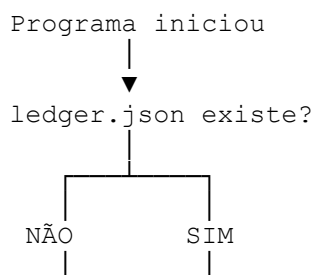
Se não existir:

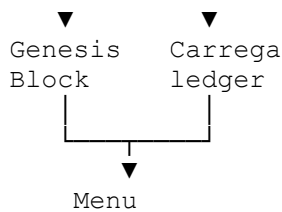
```
if os.IsNotExist(err) {
```

criamos nossa blockchain.

20. Lógica de inicialização

Nosso programa terá aproximadamente esta lógica:





Isso permite continuar exatamente de onde paramos.

21. Verificando a blockchain

Também criaremos uma função para verificar se alguém alterou os blocos.

```
func validateBlockchain(blockchain []Block) bool {  
    for i := 1; i < len(blockchain); i++ {  
        currentBlock := blockchain[i]  
        previousBlock := blockchain[i-1]  
  
        if currentBlock.PreviousHash != previousBlock.Hash {  
            return false  
        }  
  
        if calculateHash(currentBlock) != currentBlock.Hash {  
            return false  
        }  
    }  
  
    return true  
}
```

Estamos verificando duas coisas.

Primeiro:

```
currentBlock.PreviousHash != previousBlock.Hash
```

Verificamos se a corrente continua ligada.

Depois:

```
calculateHash(currentBlock) != currentBlock.Hash
```

recalculamos o hash.

Se alguém alterar:

Pagamento de R\$ 100

para:

Pagamento de R\$ 10.000

o hash calculado será completamente diferente.

22. Criando o menu

Nosso programa terá:

```
=====
                        BLOCKCHAIN GO
=====

1 - Adicionar bloco
2 - Mostrar blockchain
3 - Verificar blockchain
0 - Sair
```

Escolha:

23. Código completo

Crie o arquivo:

main.go

e coloque:

```
package main

import (
    "bufio"
    "crypto/sha256"
    "encoding/hex"
    "encoding/json"
    "fmt"
    "os"
    "strconv"
    "strings"
    "time"
)

// -----
// CONFIGURAÇÕES
// -----

// Arquivo onde nossa blockchain será armazenada.
const ledgerFile = "ledger.json"

// Quantidade de zeros exigidos no início do hash.
const difficulty = 4

// -----
// ESTRUTURA DO BLOCO
// -----

type Block struct {

    // Número do bloco.
    Index int `json:"index"`

    // Data e hora de criação.
    Timestamp string `json:"timestamp"`

    // Informação armazenada dentro do bloco.
    Data string `json:"data"`

    // Hash do bloco anterior.
    PreviousHash string `json:"previous_hash"`

    // Hash deste bloco.
```

```

    Hash string `json:"hash"`

    // Número utilizado pelo Proof of Work.
    Nonce int `json:"nonce"`
}

// -----
// CALCULAR HASH
// -----

func calculateHash(block Block) string {

    // Junta todas as informações importantes do bloco.
    record := strconv.Itoa(block.Index) +
        block.Timestamp +
        block.Data +
        block.PreviousHash +
        strconv.Itoa(block.Nonce)

    // Calcula SHA-256.
    hash := sha256.Sum256([]byte(record))

    // Converte o resultado para hexadecimal.
    return hex.EncodeToString(hash[:])
}

// -----
// PROOF OF WORK
// -----

func mineBlock(block *Block) {

    // Cria o alvo.
    //
    // Se difficulty for 4:
    //
    // target = "0000"
    target := strings.Repeat("0", difficulty)

    fmt.Println()
    fmt.Println("Minerando bloco...")

    for {

        // Calculamos um hash.
        block.Hash = calculateHash(*block)

        // Verificamos se começa com a quantidade
        // necessária de zeros.
        if strings.HasPrefix(block.Hash, target) {

            fmt.Println("Bloco minerado!")
            fmt.Println("Nonce:", block.Nonce)
            fmt.Println("Hash:", block.Hash)

            break
        }

        // Caso não seja válido, alteramos o nonce.
        block.Nonce++
    }
}

// -----

```

```

// GENESIS BLOCK
// -----

func createGenesisBlock() Block {

    fmt.Println("Criando Genesis Block...")

    block := Block{
        Index:      0,
        Timestamp:   time.Now().Format(time.RFC3339),
        Data:        "Genesis Block",
        PreviousHash: "0",
        Nonce:       0,
    }

    mineBlock(&block)

    return block
}

// -----
// CRIAR NOVO BLOCO
// -----

func createBlock(data string, previousBlock Block) Block {

    block := Block{

        // Próximo número.
        Index: previousBlock.Index + 1,

        // Momento da criação.
        Timestamp: time.Now().Format(time.RFC3339),

        // Informação digitada pelo usuário.
        Data: data,

        // Ligação com o bloco anterior.
        PreviousHash: previousBlock.Hash,

        // Começamos o Proof of Work com zero.
        Nonce: 0,
    }

    // Minera o bloco.
    mineBlock(&block)

    return block
}

// -----
// SALVAR BLOCKCHAIN
// -----

func saveBlockchain(blockchain []Block) error {

    // Converte blockchain para JSON.
    data, err := json.MarshalIndent(blockchain, "", " ")

    if err != nil {
        return err
    }

    // Grava o JSON no ledger.

```

```

    err = os.WriteFile(ledgerFile, data, 0644)

    if err != nil {
        return err
    }

    return nil
}

// -----
// CARREGAR BLOCKCHAIN
// -----

func loadBlockchain() ([]Block, error) {

    // Lê o arquivo.
    data, err := os.ReadFile(ledgerFile)

    if err != nil {
        return nil, err
    }

    var blockchain []Block

    // Converte JSON novamente para structs Go.
    err = json.Unmarshal(data, &blockchain)

    if err != nil {
        return nil, err
    }

    return blockchain, nil
}

// -----
// VERIFICAR BLOCKCHAIN
// -----

func validateBlockchain(blockchain []Block) bool {

    // Percorre a blockchain começando pelo bloco 1.
    //
    // Não começamos pelo zero porque ele é o
    // Genesis Block.
    for i := 1; i < len(blockchain); i++ {

        currentBlock := blockchain[i]

        previousBlock := blockchain[i-1]

        // Verifica se o PreviousHash realmente
        // corresponde ao hash do bloco anterior.
        if currentBlock.PreviousHash != previousBlock.Hash {

            return false
        }

        // Recalcula o hash.
        calculatedHash := calculateHash(currentBlock)

        // Verifica se alguém modificou os dados.
        if calculatedHash != currentBlock.Hash {

            return false
        }
    }
}

```



```

        }
    }

    return true
}

// -----
// MOSTRAR BLOCKCHAIN
// -----

func showBlockchain(blockchain []Block) {

    fmt.Println()
    fmt.Println("===== BLOCKCHAIN =====")

    for _, block := range blockchain {

        fmt.Println()
        fmt.Println("-----")
        fmt.Println("Bloco:", block.Index)
        fmt.Println("Timestamp:", block.Timestamp)
        fmt.Println("Dados:", block.Data)
        fmt.Println("Previous Hash:", block.PreviousHash)
        fmt.Println("Hash:", block.Hash)
        fmt.Println("Nonce:", block.Nonce)
    }

    fmt.Println("-----")
}

// -----
// MAIN
// -----

func main() {

    var blockchain []Block

    // -----
    // TENTAMOS CARREGAR O LEDGER
    // -----

    loadedBlockchain, err := loadBlockchain()

    if err != nil {

        // Se o arquivo não existe, entendemos que
        // esta é a primeira execução.
        if os.IsNotExist(err) {

            fmt.Println("Ledger não encontrado.")
            fmt.Println("Criando nova blockchain...")

            // Criamos o Genesis Block.
            genesisBlock := createGenesisBlock()

            // Colocamos o Genesis Block na blockchain.
            blockchain = append(blockchain, genesisBlock)

            // Salvamos imediatamente.
            err = saveBlockchain(blockchain)

            if err != nil {
                fmt.Println("Erro ao salvar blockchain:", err)
            }
        }
    }
}

```

```

        return
    }

    } else {

        fmt.Println("Erro ao carregar ledger:", err)
        return
    }

} else {

    // Se conseguimos ler o ledger,
    // recuperamos a blockchain.
    blockchain = loadedBlockchain

    fmt.Println("=====")
    fmt.Println("Blockchain carregada do disco.")
    fmt.Println("Blocos encontrados:", len(blockchain))
    fmt.Println("=====")
}

// Criamos um Scanner para ler o teclado.
scanner := bufio.NewScanner(os.Stdin)

// -----
// MENU PRINCIPAL
// -----

for {

    fmt.Println()
    fmt.Println("=====")
    fmt.Println("          BLOCKCHAIN GO")
    fmt.Println("=====")
    fmt.Println("1 - Adicionar bloco")
    fmt.Println("2 - Mostrar blockchain")
    fmt.Println("3 - Verificar blockchain")
    fmt.Println("0 - Sair")
    fmt.Print("Escolha: ")

    scanner.Scan()

    option := scanner.Text()

    switch option {

        // -----
        // ADICIONAR BLOCO
        // -----

        case "1":

            fmt.Print("Digite a informação do bloco: ")

            scanner.Scan()

            data := scanner.Text()

            if data == "" {
                fmt.Println("O bloco precisa possuir alguma
informação.")
            }

            continue
        }
    }
}

```

```

        // Pegamos o último bloco.
        lastBlock := blockchain[len(blockchain)-1]

        // Criamos o próximo.
        newBlock := createBlock(data, lastBlock)

        // Adicionamos à blockchain.
        blockchain = append(blockchain, newBlock)

        // Salvamos todo o ledger.
        err := saveBlockchain(blockchain)

        if err != nil {
            fmt.Println("Erro ao salvar blockchain:", err)
            continue
        }

        fmt.Println()
        fmt.Println("Bloco adicionado com sucesso!")

// -----
// MOSTRAR
// -----

case "2":

    showBlockchain(blockchain)

// -----
// VALIDAR
// -----

case "3":

    if validateBlockchain(blockchain) {

        fmt.Println()
        fmt.Println("Blockchain válida!")

    } else {

        fmt.Println()
        fmt.Println("ATENÇÃO!")
        fmt.Println("Blockchain inválida ou
modificada!")

    }

// -----
// SAIR
// -----

case "0":

    fmt.Println()
    fmt.Println("Encerrando blockchain...")
    return

default:

    fmt.Println("Opção inválida.")

}

}

```

24. Executando o projeto

Abra o PowerShell dentro da pasta:

```
go run .
```

Na primeira execução deverá aparecer algo parecido com:

```
Ledger não encontrado.  
Criando nova blockchain...
```

```
Criando Genesis Block...
```

```
Minerando bloco...
```

```
Bloco minerado!  
Nonce: 64852  
Hash: 000034f8ea...
```

```
=====
                        BLOCKCHAIN GO
=====
1 - Adicionar bloco
2 - Mostrar blockchain
3 - Verificar blockchain
0 - Sair
```

25. Adicionando nosso primeiro bloco

Escolha:

```
1
```

Digite:

```
João comprou um notebook
```

O programa começará o Proof of Work:

```
Minerando bloco...
```

Depois:

```
Bloco minerado!  
Nonce: 32714  
Hash: 00008ad75...
```

```
Bloco adicionado com sucesso!
```

26. Adicionando outro bloco

Escolha novamente:

```
1
```

Digite:

```
Maria comprou um monitor
```

Teremos:

Genesis Block

↓

Bloco 1

↓

Bloco 2

27. Visualizando nossa blockchain

Escolha:

2

Poderemos receber:

===== BLOCKCHAIN =====

Bloco: 0

Timestamp: 2026-08-17T13:30:00-03:00

Dados: Genesis Block

Previous Hash: 0

Hash: 000034f8ea...

Nonce: 64852

Bloco: 1

Timestamp: 2026-08-17T13:32:12-03:00

Dados: João comprou um notebook

Previous Hash: 000034f8ea...

Hash: 00008ad75...

Nonce: 32714

Bloco: 2

Timestamp: 2026-08-17T13:34:01-03:00

Dados: Maria comprou um monitor

Previous Hash: 00008ad75...

Hash: 00001293af...

Nonce: 7182

28. Agora vem a parte importante: persistência

Escolha:

0

O programa será encerrado.

Teremos no diretório:

blockchain-go/

```
|  
├── go.mod  
├── main.go  
└── ledger.json
```

Agora execute novamente:

```
go run .
```

Desta vez deverá aparecer:

```
=====
Blockchain carregada do disco.
Blocos encontrados: 3
=====
```

Observe que o programa **não criou outro Genesis Block**.

Ele encontrou:

```
Genesis
Bloco 1
Bloco 2
```

dentro do `ledger.json`.

29. Continuando a blockchain depois da reinicialização

Agora escolha:

```
1
```

e coloque:

```
Carlos comprou um teclado
```

Como o último bloco salvo era:

```
Bloco 2
```

o programa criará automaticamente:

```
Bloco 3
```

Nossa blockchain ficará:

```
Genesis
```

|



```
Bloco 1
```

|



```
Bloco 2
```

|



```
PROGRAMA DESLIGADO
```

|



```
PROGRAMA INICIADO NOVAMENTE
```

|



```
Bloco 3
```

Esse é um dos principais objetivos do projeto.

30. Testando adulteração

Agora podemos fazer um experimento muito interessante.

Feche o programa.

Abra:

```
ledger.json
```

Procure algo como:

```
"data": "João comprou um notebook"
```

Altere manualmente para:

```
"data": "João comprou 100 notebooks"
```

Salve.

Execute:

```
go run .
```

Escolha:

```
3 - Verificar blockchain
```

O programa deverá informar:

```
ATENÇÃO!  
Blockchain inválida ou modificada!
```

Isso acontece porque o hash salvo foi calculado utilizando:

```
João comprou um notebook
```

Mas agora temos:

```
João comprou 100 notebooks
```

Ao recalcular o SHA-256, o resultado será diferente.

31. O que está acontecendo durante a mineração?

Imagine um bloco com:

```
Dados:  
Maria pagou R$ 500
```

Nosso computador faz:

```
Nonce 0  
Hash = A8DF23...  
INVÁLIDO
```

Depois:

```
Nonce 1  
Hash = 91AD77...  
INVÁLIDO
```

Depois:

```
Nonce 2  
Hash = F83710...  
INVÁLIDO
```

E continua:

```
Nonce 3  
Nonce 4  
Nonce 5  
Nonce 6  
...  
Nonce 18452
```

Até encontrar:

```
0000A291FF...
```

Como começa com:

```
0000
```

o Proof of Work terminou.

32. O papel da dificuldade

Utilizamos:

```
const difficulty = 4
```

Experimente mudar para:

```
const difficulty = 3
```

A mineração deverá ficar mais rápida.

Depois experimente:

```
const difficulty = 5
```

Ela deverá levar mais tentativas.

Isso acontece porque encontrar:

```
00000...
```

é mais difícil do que encontrar:

```
000...
```


33. Conceitos de Go utilizados neste projeto

Este projeto reúne grande parte dos assuntos estudados durante um curso introdutório/intermediário de Go.

Utilizamos:

Structs

```
type Block struct {  
}
```

Slices

```
[]Block
```

Funções

```
func mineBlock() {  
}
```

Ponteiros

```
func mineBlock(block *Block)
```

Condições

```
if err != nil {  
}
```

Loops

```
for {  
}
```

Range

```
for _, block := range blockchain {  
}
```

Switch

```
switch option {  
}
```

Arquivos

```
os.ReadFile()  
os.WriteFile()
```

JSON

```
json.MarshalIndent()  
json.Unmarshal()
```

Tratamento de erros

```
if err != nil {  
}
```

Entrada pelo teclado

```
bufio.NewScanner(os.Stdin)
```

Pacotes de criptografia

```
crypto/sha256
```

Conversões

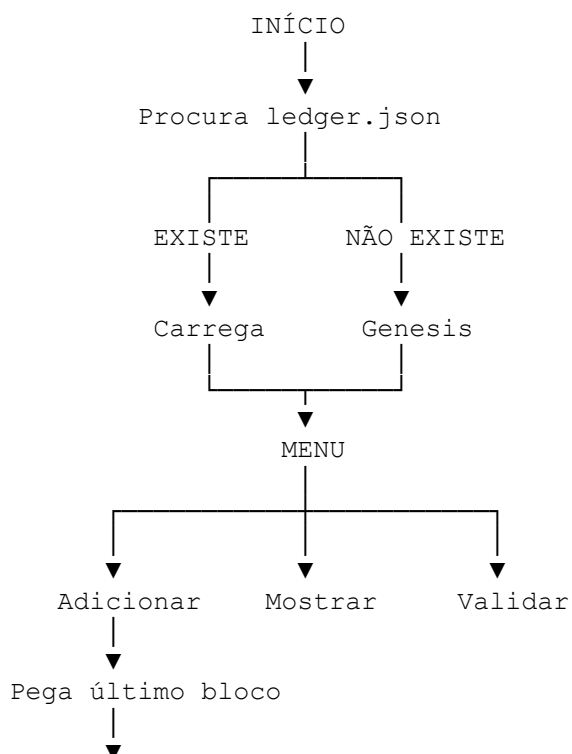
```
strconv.Itoa()
```

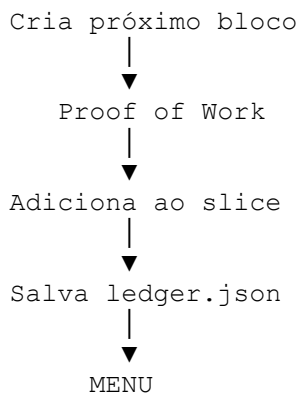
Strings

```
strings.HasPrefix()
```

34. Fluxo completo da aplicação

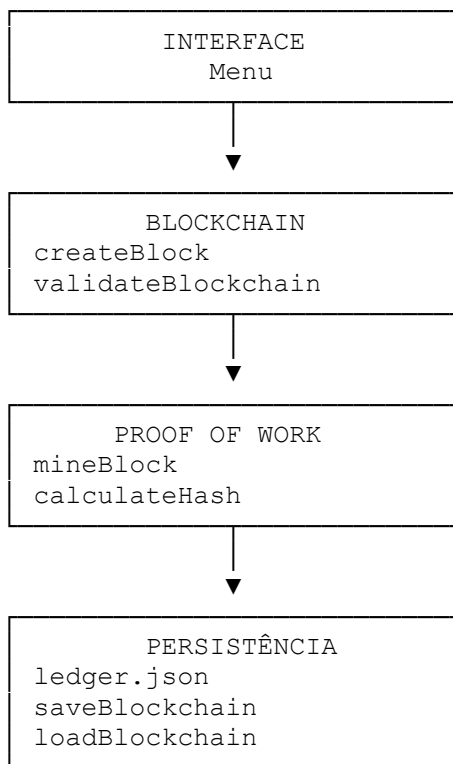
Nosso sistema funciona aproximadamente assim:





35. Arquitetura conceitual

Podemos enxergar nosso pequeno sistema dividido em quatro partes.



36. Um detalhe importante: isso é uma blockchain educacional

Nosso projeto demonstra diversos conceitos reais encontrados em blockchains:

- Blocos
- Hashes
- Encadeamento
- Genesis Block
- Nonce

- Proof of Work
- Dificuldade
- Validação
- Ledger
- Persistência

Porém ele ainda **não é uma blockchain distribuída como Bitcoin.**

Nossa blockchain possui apenas:

1 computador

Não possuímos:

Node A
Node B
Node C
Node D

conversando pela rede.

Também não implementamos:

- Transações assinadas digitalmente
- Chaves públicas e privadas
- Carteiras
- Comunicação P2P
- Consenso entre vários nós
- Merkle Tree
- Recompensa de mineração
- Moeda digital
- Ajuste automático de dificuldade

Tudo isso poderia aparecer em versões futuras do projeto.

37. Desafio 1: adicionar o nome do minerador

Adicione:

```
Miner string `json:"miner"`
```

ao bloco.

O programa deverá perguntar:

Nome do minerador:

O resultado poderá ser:

```
Bloco: 5  
Minerador: Antonio  
Nonce: 57381  
Hash: 00002af...
```

38. Desafio 2: registrar transações

Em vez de permitir qualquer texto:

Digite a informação:

mude para:

Remetente:

João

Destinatário:

Maria

Valor:

500

Podemos futuramente criar:

```
type Transaction struct {  
    From    string  
    To      string  
    Amount  float64  
}
```

Então o bloco poderia possuir:

```
Transaction Transaction
```

39. Desafio 3: mostrar tempo de mineração

Podemos medir quanto tempo o computador levou.

Antes da mineração:

```
start := time.Now()
```

Depois:

```
duration := time.Since(start)
```

Mostrando:

```
fmt.Println("Tempo de mineração:", duration)
```

O aluno poderá comparar:

```
difficulty = 3  
difficulty = 4  
difficulty = 5
```

40. Desafio 4: contar tentativas

O próprio nonce nos dá aproximadamente a quantidade de tentativas.

Podemos mostrar:

```
fmt.Println("Tentativas:", block.Nonce+1)
```

Exemplo:

Minerando bloco...

Bloco minerado!

Tentativas: 48382

Nonce: 48381

Hash:

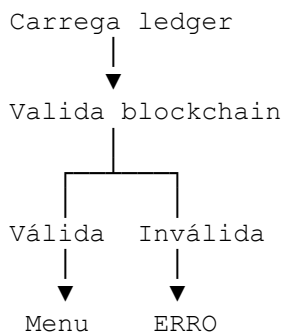
00007be92009...

41. Desafio 5: proteger a inicialização

Existe uma melhoria importante que pode ser feita.

Atualmente carregamos o ledger e depois permitimos utilizar o programa.

Uma versão melhor deveria fazer:



Assim, uma blockchain adulterada não seria utilizada.

Por exemplo:

```
if !validateBlockchain(blockchain) {
    fmt.Println("ERRO: blockchain armazenada é inválida.")
    return
}
```

42. Desafio final do projeto

Depois de construir a versão apresentada nesta aula, tente transformar cada registro do bloco em uma transferência financeira.

O sistema poderia possuir o menu:

```
=====
                        GO BLOCKCHAIN
=====
```

```
1 - Nova transação
2 - Minerar bloco
3 - Mostrar blockchain
4 - Validar blockchain
5 - Mostrar último bloco
6 - Informações da rede
0 - Sair
```

Uma transação poderia possuir:

Origem: Alice
Destino: Bob
Valor: 150.00

O bloco armazenaria:

Bloco 7

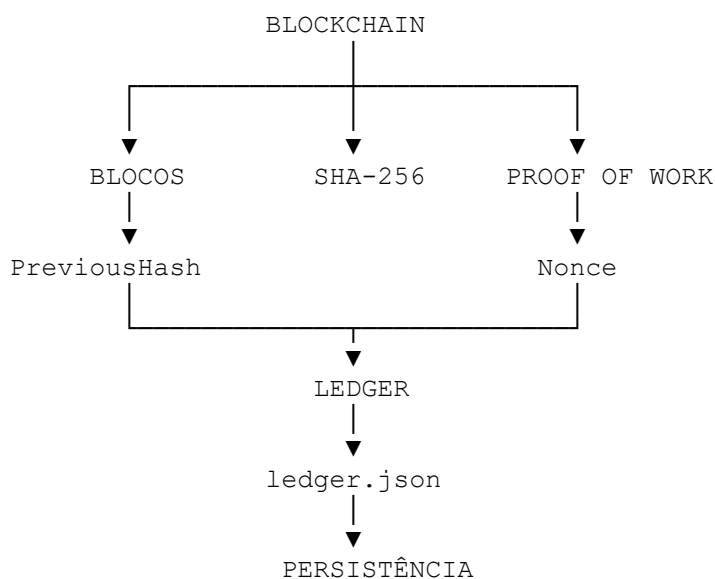
Alice
↓ R\$150
▼
Bob

Esse seria um excelente ponto de partida para posteriormente estudar:

Carteiras
↓
Criptografia assimétrica
↓
Chaves públicas
↓
Chaves privadas
↓
Assinaturas digitais
↓
Transações
↓
Rede P2P
↓
Blockchain distribuída

43. Resultado final

Ao final deste projeto construímos uma pequena blockchain funcional com:



Quando o programa é executado pela primeira vez:

Não existe ledger
↓

Cria Genesis Block
↓
Minera
↓
Salva ledger.json

Nas próximas execuções:

Existe ledger.json
↓
Carrega blockchain
↓
Recupera último bloco
↓
Continua blockchain

Portanto, se tivermos:

Bloco 0 → Bloco 1 → Bloco 2 → Bloco 3

e desligarmos o programa, esses registros permanecem no disco.

Quando o sistema for iniciado novamente:

Bloco 0 → Bloco 1 → Bloco 2 → Bloco 3
↓
Bloco 4
↓
Bloco 5

A blockchain continua exatamente de onde havia parado.

Esse projeto é especialmente interessante como conclusão de um curso de Go porque transforma vários conceitos estudados separadamente em uma aplicação única: **estruturas, ponteiros, slices, loops, funções, arquivos, JSON, erros, criptografia, persistência e algoritmos trabalhando juntos em um sistema completo.**